

SQL Exam Cheat

Sheet

סדר ביצוע השאילתה-

```
SELECT
[ALL | DISTINCT | DISTINCTROW ]
select_expr [, select_expr] ...
[FROM table_references
[WHERE where_condition]
[GROUP BY {col_name | expr | position}
[HAVING where_condition]
[ORDER BY {col_name | expr | position}
[ASC | DESC], ...]
[LIMIT [{offset}, row_count | row_count OFFSET offset];
```

SELECT פקודות בסיסיות

Command	משמעות	תחביר
SELECT	בחירת עמודות	SELECT col1,col2
FROM	מאין טבלה	FROM table
WHERE	סינון שורות	WHERE condition
DISTINCT	הסרת כפילויות	SELECT DISTINCT col
ORDER BY	מיון תוצאות	ORDER BY col ASC/DESC
LIMIT	הגבלת מספר תוצאות	LIMIT n

WHERE-מסמן שורות לפני גרופי-
SELECT ...
FROM table
WHERE condition;

HAVING-סימן אחרי גרופי-
SELECT director_id,
COUNT(*) AS movies_count
FROM movies_directors
GROUP BY director_id
HAVING COUNT(*) >= 10;

חשוב לשים לב-
WHERE → לפני GROUP BY
HAVING → אחרי GROUP BY

Operator	משמעות	דוגמה
=	שווה	WHERE year = 2000
!= / <>	לא שווה	WHERE genre <> 'Drama'
>	גדול מ	WHERE rank > 8
<	קטן מ	WHERE year < 1950
>=	גדול או שווה	WHERE rank >= 9
<=	קטן או שווה	WHERE year <= 2000
BETWEEN	בין ערכים	WHERE year BETWEEN 1990 AND 1999
NOT BETWEEN	לא בין ערכים	WHERE year NOT BETWEEN 1990 AND 1999
LIKE 'Titanic'	בדיוק הערך 'Titanic'	WHERE name LIKE 'Titanic'
LIKE 'A%'	מתחיל ב A	WHERE name LIKE 'Star%'
LIKE '%A'	מסתיים ב A	WHERE name LIKE '%Man'
LIKE '%A%'	מכיל A	WHERE name LIKE '%Love%'
LIKE ' _ A%'	תו אחד ואז A	WHERE name LIKE ' _ at%'
IN (...)	נמצא ברשימה	WHERE genre IN ('Comedy','Drama')
NOT IN (...)	לא ברשימה	WHERE genre NOT IN ('Comedy','Drama')
IS NULL	ערך חסר	WHERE rank IS NULL
IS NOT NULL	ערך קיים	WHERE rank IS NOT NULL
OR	לפחות תנאי אחד	WHERE year = 2000 OR year = 2001
AND	כל התנאים	WHERE year >= 2000 AND rank > 8
NOT	היפוך תנאי	WHERE NOT year = 2000

Function	משמעות	תחביר + דוגמה
LENGTH(col)	מספר תווים	SELECT LENGTH('Titanic') → 7
UPPER(col)	אותיות גדולות	SELECT UPPER('Titanic') → 'TITANIC'
LOWER(col)	אותיות קטנות	SELECT LOWER('Titanic') → 'titanic'
LEFT(col,n)	תווים ראשונים n	SELECT LEFT('Titanic',4) → 'Tita'
RIGHT(col,n)	תווים אחרונים n	SELECT RIGHT('Titanic',3) → 'tic'
SUBSTR(col,s,l)	תווים מתחיל s עד l	SELECT SUBSTR('Titanic',2,4) → 'itan'
CONCAT(a,b,...)	חיבור תחתיות	SELECT CONCAT('James',' ', 'Cameron') → 'James Cameron'
TRIM(col)	מסיר רווחים מותרים בתחילת ובסוף הטקסט	SELECT TRIM('Titanic ') → 'Titanic'
ROUND(num,d)	ספירת אחרי הנקודה לעיגול מספר ל d	SELECT ROUND(8.678,2) → 8.68

Function	משמעות	תחביר + דוגמה
COUNT(*)	סופר שורות	SELECT COUNT(*) FROM movies → 388269
COUNT(col)	ערבים שאינם NULL	SELECT COUNT(rank) FROM movies → דירוג מספר הסרטים עם דירוג
COUNT(DISTINCT col)	ערבים שונים	SELECT COUNT(DISTINCT year) FROM movies → מספר השנים השונות
SUM(col)	סכום	SELECT SUM(rank) FROM movies → סכום כל הדירוגים
AVG(col)	מומוצע	SELECT AVG(rank) FROM movies → דירוג ממוצע
MAX(col)	מקסימום	SELECT MAX(rank) FROM movies → הדירוג הגבוה ביותר
MIN(col)	מינימום	SELECT MIN(rank) FROM movies → הדירוג הנמוך ביותר
GROUP_CONCAT(col)	איחוד למחרוזת	GROUP_CONCAT(role) → 'Hero,Villain,Detective'
GROUP_CONCAT(DISTINCT col)	איחוד ללא כפילויות	GROUP_CONCAT(DISTINCT role) → 'Hero,Villain'
GROUP_CONCAT(DISTINCT col ORDER BY col)	איחוד ללא כפילויות בסדר קבוע	GROUP_CONCAT(DISTINCT role ORDER BY role) → 'Detective,Hero,Villain'

LEFT JOIN	Everything on the left + anything on the right that matches	<pre>SELECT * FROM TABLE_1 LEFT JOIN TABLE_2 ON TABLE_1.KEY = TABLE_2.KEY</pre>
RIGHT JOIN	Everything on the right + anything on the left that matches	<pre>SELECT * FROM TABLE_1 RIGHT JOIN TABLE_2 ON TABLE_1.KEY = TABLE_2.KEY</pre>
INNER JOIN	Only the things that match on the left AND the right	<pre>SELECT * FROM TABLE_1 INNER JOIN TABLE_2 ON TABLE_1.KEY = TABLE_2.KEY</pre>

GROUP BY- מחלק את הרשומות לקבוצות לפי עמודה ומחשב אגריגציה לכל קבוצה-

```
SELECT col,
AGG_FUNC(...)
FROM table
GROUP BY col;

לפי כמה עמודות = יוצר קבוצה עבור כל צירוף של 2 העמודות-
SELECT col1, col2,
AGG_FUNC(...)
FROM table
GROUP BY col1, col2;

דוגמאות- כמה סרטים יש לכל במאי-
SELECT director_id,
COUNT(*) AS movies_count
FROM movies_directors
GROUP BY director_id;

דירוג ממוצע של סרטים בכל שנה-
SELECT year,
AVG(rank) AS avg_rank
FROM movies
GROUP BY year;
```

שאילתה שמקבלת שם וניתן להשתמש בה כמו טבלה - **View**
זה שומר את השאילתה, לא את התוצאות

תחביר-

CREATE VIEW v AS

SELECT ...

שימוש-

SELECT *

FROM v;

זמני עבור שאילתה אחת **CTE** -

תחביר-

WITH t AS (SELECT ...)

SELECT *

FROM t;

DML-

Command	משמעות	תחביר + דוגמה
INSERT	הוספת שורות	INSERT INTO my_movies VALUES (1,'Titanic',1997,8.0)
INSERT ... SELECT	הוספת נתונים משאילתה	INSERT INTO my_movies SELECT * FROM movies WHERE year = 2000
UPDATE	עדכון שורות	UPDATE tmp_movies SET year = year + 1 WHERE id >= 0
DELETE	מחיקת שורות	DELETE FROM tmp_movies WHERE id < 100

Case when-

תחביר + דוגמה	הסבר	משא
CASE WHEN rank >= 8 THEN 'Excellent' ELSE 'Not excellent' END AS rating_group	תנאי בתוך SELECT. כמו if/else	CASE
WHEN rank >= 8	התנאי שבדקים	WHEN
THEN 'Excellent'	מה להחזיר אם התנאי נכון	THEN
ELSE 'Other'	מה להחזיר אם אף תנאי לא התקיים	ELSE
END AS rating_group	סוגר את הCASE	END

דוגמה- אם הדירוג גדול מ 8, excellent =, אם הדירוג גדול מ 6, good =, אם הדירוג null rank =

```
SELECT
name,
rank,
CASE
WHEN rank >= 8 THEN 'Excellent'
WHEN rank >= 6 THEN 'Good'
WHEN rank IS NULL THEN 'No rank'
ELSE 'Low'
END AS rank_group
FROM movies;
```

Window Functions – מחשב ערך לכל שורה בעזרת שורות נוספות בהקשר שלה –

תחביר כללי-

FUNCTION(...) OVER (

פונקציות חשובות-

Function	משמעות
ROW_NUMBER()	מספר שורות ייחודי
RANK()	דירוג עם קפיצות
DENSE_RANK()	דירוג ללא קפיצות
LAG()	הערך מהשורה הקודמת
LEAD()	הערך מהשורה הבאה
AVG() OVER	ממוצע בחלון
SUM() OVER	סכום בחלון
COUNT() OVER	ספירה בחלון

כל שורה מקבלת מספר ROW_NUMBER =

דירוג אמיתי בתחרות - אם שניים מקום 2 אז הבא הוא מקום 4 RANK =

לא משאיר חורים, אם שניים מקום 2 אז הבא מקום 3 DENSE_RANK =

משתמשים בשתוצאה של שאילתה אחת נדרשת לשאילתה אחרת- **Subqueries**

תחביר כללי-

SELECT ...

FROM (

SELECT ...

) AS t;

חיבור של טבלה עם עצמה כדי להשוות רשומות מאותה טבלה – **SELF JOIN**

```
SELECT *
FROM ____
JOIN ____
ON ____
WHERE ____ < ____;

(A,A), (B,A) :מסיר זוגות זהים וגם כפילויות הפוכות
(A,B) :משאיר רק
למצוא את כל מי שאין לו ... LEFT JOIN + NULL-
SELECT A.*
FROM A
LEFT JOIN B
ON ...
WHERE B.id IS NULL;
```

Data base design-

דוגמה מהIMDb	הגדרה	מושג
Movie, Actor, Director	אובייקט שעליו שומרים מידע	Entity (ישות)
Movie: id, name, year, rank	תכונה של ישות	Attribute (מאפיין)
Actor ↔ Movie, Director ↔ Movie	קשר בין שתי ישויות	Relationship (קשר)
1:1, 1:N, N:N	כמה מופעים של ישות אחת יכולים להיות קשורים לישות אחרת	Cardinality (עוצמת קשר)

Cardinality-

מימוש במסד	דוגמה	משמעות	Cardinality
FK באחת הטבלאות	Person ↔ Passport לאדם אחד יש id אחד	ישות אחת קשורה לישות אחת	1:1
N בצד ה FK (movies.director_id)	Director → Movies במאי אחד בהרבה סרטים	ישות אחת קשורה למספר ישויות	1:N
טבלת קשר(roles) roles-> actor_id, movie_id, role	Actors ↔ Movies כמה שחקנים בכמה סרטים	מספר ישויות קשורות למספר ישויות	N:N

Examples-

1:1 movies -

CREATE TABLE movies (
id INT PRIMARY KEY,
name VARCHAR(100),
year INT CHECK (year >= 1900),
rank FLOAT,
);

1:N Directors → Movies -

CREATE TABLE movies (
id INT PRIMARY KEY,
name VARCHAR(100),
director_id INT,
FOREIGN KEY (director_id)
REFERENCES directors(id)
);

N:N Actors ↔ Movies –

CREATE TABLE roles (
actor_id INT,
movie_id INT,
role VARCHAR(100),
PRIMARY KEY (actor_id, movie_id),
FOREIGN KEY (actor_id) REFERENCES actors(id),
FOREIGN KEY (movie_id) REFERENCES movies(id)
);

שמירה על נכונות ועקביות הנתונים - **Data Integrity**
המטרה- שהמידע יהיה אמין, עדכני ונכון.
בעיות שרוצים למנוע-
חזרתיות- אותו מידע נשמר פעמיים
חוסר עקביות- עדכונים לא תואמים
נתונים חסרים – שדות ריקים
ערכים לא תקינים- נתונים בפורמט שגוי
כפילויות- רשימות חוזרות

- שמירה על data integrity נעשית באמצעות הגבלות -

חוקים שהמסד אוכף אוטומטית כדי למנוע הכנסת נתונים לא תקינים - **constraints**

Constraint	משמעות	דוגמה
NOT NULL	חייב להיות ערך	name VARCHAR(100) NOT NULL
UNIQUE	אין כפילויות	email VARCHAR(100) UNIQUE
PRIMARY KEY	UNIQUE + NOT NULL	id INT PRIMARY KEY
FOREIGN KEY	הערך חייב להופיע בטבלה אחרת	FOREIGN KEY(movie_id) REFERENCES movies(id)
CHECK	הערך חייב לעמוד בתנאי מסוים	CHECK (year >= 1900)
DEFAULT	ערך ברירת מחדל	status VARCHAR(20) DEFAULT 'Active'
TYPE	מגביל את סוג הנתונים	year INT, name VARCHAR(100)

NULL:- unknown or missing
Check NULL only with: IS NULL / IS NOT NULL
Never use: = NULL

"יצוג נתונים + Data Types

Type	שימוש	מתאים ל...	דוגמה
INT	מספר שלם	id, year, age	year INT
FLOAT	מספר עשיוני בקירוב	rank, ממוצעים	rank FLOAT
NUMERIC(p,s)	מספר עשרוני מדויק	כסף, מחירים	price NUMERIC(8,2)
DECIMAL(p,s)	מספר עשרוני מדויק	כסף, משכורות	salary DECIMAL(10,2)
CHAR(n)	טקסט באורך קבוע	קודי מדינה, ת"ז בפורמט קבוע	country_code CHAR(2)
VARCHAR(n)	טקסט באורך משתנה	שמות, כתובות, אימיילים	name VARCHAR(100)
DATE	תאריך	יום הולדת, תאריך יציאה	release_date DATE
TIME	שעה	שעת התחלה/סיום	start_time TIME
DATETIME	תאריך ושעה	אירועים, הזמנות	created_at DATETIME
TIMESTAMP	חותמת זמן	מעקב עדכונים	updated_at TIMESTAMP
BOOLEAN	אמת / שקר	דגלים לוגיים	is_active BOOLEAN

Use the smallest suitable type
Bad:age VARCHAR(10)
Good:age INT

Flexible Schemas - מבנה נתונים אינו קבוע מראש -
מבנה נתונים גמיש = JSON
key : value - מורכב מזוגות
יצירת טבלה גמישה-
CREATE TABLE ()
הכנסת סרט לטבלה-
INSERT INTO TABLE_NAME VALUES

ארגון הנתונים בטבלאות כדי לצמצם כפילויות ותלות מיותרת. נעשה זאת ע"י פירוק טבלאות גדולות - **Normalization**
עם מידע חוזר לטבלאות קטנות יותר עם קשרים בינהן
מניעת חוסר עקביות,מניעת כפילויות :מטרות

Form	כלל
1NF	כל תא ערך אחד, אין כפילויות
2NF	כל עמודה שאינה מפתח ראשי צריכה להיות תלויה בכל המפתח הראשי
3NF	אין תלות בין עמודות שאינן מפתח